



Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre

A Project Report
on
“Polynomial Interpolation and Numerical Integration using
Classical Numerical Methods”

[Code No.: MCSC 202]
(For partial fulfillment of Year II / Semester II in Computer Engineering)

Submitted by
Safal Bastola (037960-24)
Asmit Panthi (037988-24)
Sangam Paudel (037989-24)

Submitted to
Dr. Sushil Ghimire
Department of Mathematics

August 3, 2026

Bona fide Certificate

This project work on
“Polynomial Interpolation and Numerical Integration using Classical
Numerical Methods”
is the bona fide work of
“
Safal Bastola (037960-24),
Asmit Panthi (037988-24),
Sangam Paudel (037989-24)
”
who carried out the project work under my supervision.

Project Supervisor

Dr. Sushil Ghimire
Assistant Professor
Department of Mathematics

Acknowledgements

We would like to express our sincere gratitude to Dr. Sushil Ghimire, the course instructor for MCSC 202 Numerical Methods, for the clear lectures and worked examples that made the derivation and verification of these methods approachable, and for the guidance that shaped this assignment. The lab sessions on polynomial interpolation and numerical integration directly informed the way the algorithms in this report were implemented and tested. We are also thankful to the Department of Computer Science and Engineering, Kathmandu University, for providing the learning environment and computing resources used to carry out this work. Finally, we are grateful to our classmates for the discussions that helped us reason through the behaviour of the methods, and to our family for their continued support throughout the semester.

Abstract

Numerical methods provide systematic ways of obtaining approximate or verifiable solutions to mathematical problems that are difficult, tedious, or impossible to solve by purely closed-form means. This report presents the theory, implementation, and analysis of four classical numerical techniques applied to two representative problems assigned from the MCSC 202 Numerical Methods course. The first problem is the construction of the unique cubic polynomial interpolating the data $x = (1, 2, 4, 7)$, $y = (3, 6, 5, 10)$, using Newton's Divided Difference method and the Lagrange method, with both forms evaluated at $x = 5$ and cross-checked against SciPy's built-in interpolation routine. The second problem is the evaluation of the definite integral $\int_0^2 (x^3 + 2x + 1) dx$ using the composite Trapezoidal Rule and the composite Simpson's 1/3 Rule, with both approximations compared against the analytically computed exact value 10, obtained symbolically using SymPy.. All four methods were implemented in Python 3 using NumPy for array computation, and the results were verified and visualised using Matplotlib. The experiments confirm the theoretical expectations: the Newton and Lagrange forms expand to the identical polynomial $P_3(x) = \frac{4}{15}x^3 - \frac{91}{30}x^2 + \frac{307}{30}x - \frac{67}{15}$ and agree at $x = 5$ to machine precision, exactly as the uniqueness theorem for polynomial interpolation predicts; and Simpson's 1/3 Rule integrates the cubic integrand exactly at every choice of subintervals n , whereas the Trapezoidal Rule exhibits a second-order $O(h^2)$ error that falls by a factor of four each time n is doubled. The report demonstrates that correctly checking a numerical result, whether through an algebraic uniqueness argument, an exact reference value, or an independent library routine, is as important as computing the result in the first place, and that even elementary problems reveal clear practical differences between competing numerical schemes.

Keywords: *Polynomial Interpolation, Newton's Divided Difference Method, Lagrange Interpolation, Numerical Integration, Trapezoidal Rule, Simpson's 1/3 Rule, Python*

Contents

Acknowledgements	ii
Abstract	iii
List of Figures	vi
List of Tables	vii
List of Symbols and Abbreviations	viii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Limitations	2
1.5 Tools Used	3
2 Polynomial Interpolation of a Data Set	4
2.1 Theoretical Background	4
2.1.1 The Polynomial Interpolation Problem	4
2.1.2 Newton's Divided Difference Method	4
2.1.3 The Lagrange Method	5
2.1.4 Equivalence of the Two Forms	5
2.2 Python Implementation	5
2.3 Results	7
2.3.1 Divided Difference Table	7
2.3.2 Newton and Lagrange Polynomials	7
2.3.3 Evaluation at $x = 5$	7
2.4 Graphical Analysis	8
2.5 Discussion	9
3 Numerical Integration of a Polynomial	10
3.1 Theoretical Background	10
3.1.1 The Problem of Numerical Integration	10
3.1.2 The Composite Trapezoidal Rule	10
3.1.3 The Composite Simpson's 1/3 Rule	11
3.1.4 Exact Value of the Integral	11
3.2 Python Implementation	12
3.3 Results and Output	13
3.4 Graphical Analysis	13
3.5 Discussion	15

4 Conclusion	16
4.1 Future Work	16
References	18
Appendix A	19
A.1 Polynomial Interpolation Program (question7.py)	19
A.2 Numerical Integration Program (question12.py)	22

List of Figures

2.1	The Newton and Lagrange interpolating polynomials through the data $(1, 3), (2, 6), (4, 5), (7, 10)$, with the evaluation point $x = 5$ marked.	8
2.2	Absolute difference between the Newton and Lagrange forms evaluated over the same range of x . The difference never exceeds machine round-off, confirming the two methods construct the same interpolating polynomial.	9
3.1	The integrand $f(x) = x^3 + 2x + 1$ and the trapezoidal approximation of the area for $n = 4$. The trapezium tops lie above the convex curve, so the rule overestimates the true area.	14
3.2	Absolute error versus the number of subintervals n on log–log axes. The Trapezoidal error decreases with slope -2 ($O(h^2)$); Simpson’s error stays at the machine round-off floor because the rule is exact for cubics.	14

List of Tables

2.1	Divided-difference table for the data $x = (1, 2, 4, 7)$, $y = (3, 6, 5, 10)$.	7
3.1	Absolute error of the two rules for the integral $\int_0^2 (x^3 + 2x + 1) dx$ (exact value = 10).	13

List of Symbols and Abbreviations

$\prod$ Product operator indicating the multiplication of a sequence of terms..

$\sum$ Summation symbol indicating the addition of a sequence of terms.

ξ A point in the interval (a, b) , typically appearing in error or remainder terms of numerical methods..

a, b lower and upper limits of integration.

$f(x)$ the function under study (the integrand, or the underlying relationship interpolated by the data).

$f[x_0, \dots, x_k]$ k -th order divided difference of the data over the nodes $x_0, \dots, x_k$.

h step size / width of a subinterval, $h = (b - a)/n$.

n number of subintervals (integration) or the interpolation degree (interpolation).

$P_n(x)$ interpolating polynomial of degree at most n .

x_i i -th data node used for interpolation.

DD Divided Difference.

LP Lagrange Polynomial.

NM Numerical Method.

ODE Ordinary Differential Equation.

Chapter 1

Introduction

This chapter introduces the two numerical problems addressed in this report, explains why numerical methods are needed even for problems with a simple closed form, and states the objectives, scope, and tools used throughout the work.

1.1 Background

Two mathematical tasks recur throughout engineering and applied science: constructing a function that passes through a given set of measured or tabulated data points, and evaluating a definite integral. When only a finite table of (x, y) pairs is available, there is no single formula for the underlying relationship, so a polynomial is fitted through the known points and used to estimate the function elsewhere; when the integrand is complicated or only known at discrete points, its antiderivative may be difficult or impossible to write down, so the definite integral is instead approximated from a finite number of function evaluations. Both tasks are examples of *numerical methods*: finite, mechanical sequences of arithmetic operations that a computer can carry out and that are designed to reproduce or approximate the exact mathematical answer.

This report deliberately uses two problems for which independent checks are available. The first problem is the construction of the unique cubic polynomial that interpolates the four data points

$$x = (1, 2, 4, 7), \quad y = (3, 6, 5, 10), \quad (1.1)$$

built by two different methods, Newton's Divided Difference method and the Lagrange method, whose results can be checked against each other and against SciPy's built-in interpolation routine. The second problem is the definite integral

$$I = \int_0^2 (x^3 + 2x + 1) dx, \quad (1.2)$$

evaluated using the Trapezoidal Rule and Simpson's 1/3 Rule and compared against the exact value obtained symbolically. Because the exact interpolating polynomial and the exact integral are both obtainable independently, these problems are well suited to studying the accuracy, agreement, and computational behaviour of the methods rather than merely producing a number.

1.2 Problem Statement

The assignment poses two independent computational tasks.

- **Polynomial Interpolation (Question 7).** Write a Python program that, given the data of Equation (1.1), constructs the interpolating polynomial using both Newton's Divided Difference method and the Lagrange method, evaluates both polynomials at $x = 5$, and verifies that the two methods agree.
- **Numerical Integration (Question 12).** Write a Python program that evaluates the integral in Equation (1.2) numerically using the Trapezoidal Rule and Simpson's 1/3 Rule. The program must accept the number of subintervals n from the user, compute the numerical value by each method, compute the exact value, calculate the absolute error of each method, and display the results in a tabular format.

1.3 Objectives

The following objectives were set for this work.

- To study the theory behind Newton's Divided Difference method, the Lagrange method, the Trapezoidal Rule, and Simpson's 1/3 Rule, including the derivation of each formula.
- To implement all four methods in Python 3, using NumPy, SciPy, and SymPy where appropriate, so that the logic in the code mirrors the underlying theory.
- To verify each numerical result independently: the two interpolation methods against each other and against SciPy's built-in routine, and the two integration rules against the exact value obtained symbolically.
- To visualise the behaviour of the methods with clearly labelled graphs, in particular the agreement between the two interpolation forms and the way the integration error decreases as more subintervals are used.
- To compare the methods within each problem and draw conclusions about accuracy, computational convenience, and appropriate use.

1.4 Scope and Limitations

The report is limited to the two specific problems stated above and to the four methods named in the questions; it is not a general survey of interpolation or numerical integration. The interpolation study uses a small, fixed set of four data points and a single evaluation point that lies inside the span of the data, so extrapolation behaviour and the practical difficulties of high-degree interpolation with many nodes (Runge's phenomenon) are outside its scope. The integration study uses a polynomial integrand, which is convenient because Simpson's rule is exact for it, but this also means the round-off floor of Simpson's rule is reached almost immediately and its error cannot be studied as a function of n in the same way as the Trapezoidal Rule. The programs use ordinary double-precision floating-point arithmetic, so all reported errors below roughly 10^{-15} are dominated by machine round-off rather than by the method itself.

1.5 Tools Used

All programs were written in Python 3 (Python Software Foundation, 2024). The scientific array library NumPy (Harris et al., 2020) was used for numerical arrays and vectorised computation, SymPy was used to obtain exact symbolic polynomials and the exact symbolic value of the integral, SciPy’s `scipy.interpolate.lagrange` was used as an independent cross-check on the interpolation results, and Matplotlib (Hunter, 2007) was used to produce every figure in this report. The report itself was typeset in L^AT_EX.

Chapter 2

Polynomial Interpolation of a Data Set

This chapter addresses Question 7. It develops the theory of Newton's Divided Difference method and the Lagrange method for polynomial interpolation, explains why the two methods must agree, presents the Python implementation, reports the computed divided-difference table and interpolating polynomials, illustrates the result graphically, and discusses the practical differences between the two forms.

2.1 Theoretical Background

2.1.1 The Polynomial Interpolation Problem

Given $n + 1$ data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ with distinct abscissae x_i , the interpolation problem asks for a function that passes exactly through every point. Restricting the search to polynomials of degree at most n , the following existence and uniqueness result underlies the whole chapter.

There exists exactly one $P_n(x)$ of degree $\leq n$ with $P_n(x_i) = y_i$ for $i = 0, \dots, n$.
(2.1)

Existence and uniqueness follow from the non-singularity of the Vandermonde matrix built from the distinct nodes x_i : the $n + 1$ interpolation conditions form a linear system in the $n + 1$ unknown coefficients of P_n whose determinant is the Vandermonde determinant $\prod_{i < j} (x_j - x_i)$, which is non-zero whenever the nodes are distinct. Equation (2.1) is the reason the Newton and Lagrange methods described below, although built from completely different formulas, are guaranteed to produce the *same* polynomial: both are simply different bookkeeping schemes for constructing the one interpolant that Equation (2.1) says must exist.

For this problem the data set is

$$x = (1, 2, 4, 7), \quad y = (3, 6, 5, 10), \quad (2.2)$$

so $n = 3$ and the interpolant $P_3(x)$ is a cubic polynomial.

2.1.2 Newton's Divided Difference Method

Newton's method builds the interpolating polynomial incrementally using divided differences. The zeroth-order divided difference of a point is simply its function value, $f[x_i] = y_i$; higher-order divided differences are defined recursively by

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}. \quad (2.3)$$

Arranging these in a triangular table, the entries along the top diagonal, $f[x_0]$, $f[x_0, x_1]$, $f[x_0, x_1, x_2]$, $f[x_0, x_1, x_2, x_3]$, become the coefficients of the Newton form of the interpolating polynomial,

$$\begin{aligned} P_3(x) = & f[x_0] + f[x_0, x_1](x - x_0) \\ & + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ & + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2). \end{aligned} \quad (2.4)$$

Each successive term corrects the previous, lower-degree approximation, which makes the Newton form convenient when an extra data point is added later: the earlier coefficients do not need to be recomputed, only one new diagonal entry.

2.1.3 The Lagrange Method

The Lagrange method constructs the same interpolant directly, without an intermediate table, using the Lagrange basis polynomials

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}, \quad i = 0, 1, \dots, n. \quad (2.5)$$

Each basis polynomial L_i is constructed to satisfy $L_i(x_k) = 1$ when $k = i$ and $L_i(x_k) = 0$ for every other node x_k , so the combination

$$P_n(x) = \sum_{i=0}^n y_i L_i(x) \quad (2.6)$$

automatically satisfies $P_n(x_i) = y_i$ at every node. Unlike the Newton form, the Lagrange form requires no table and gives an explicit formula, but every basis polynomial must be rebuilt from scratch if even a single data point is added or changed.

2.1.4 Equivalence of the Two Forms

Because Equation (2.1) guarantees a unique degree- $\leq n$ polynomial through the given points, Equations (2.4) and (2.6) must expand to the *same* polynomial in x , even though they are built from unrelated-looking formulas and are evaluated by different arithmetic. This equivalence is exactly what is checked numerically in Section 2.3.

2.2 Python Implementation

The program was written in Python 3 using NumPy for array handling, SymPy for the symbolic (exact-coefficient) form of each polynomial, and SciPy's `scipy.interpolate.lagrange` as an independent cross-check. Listing 2.1 shows the divided-difference table builder and the Newton evaluation routine, which implement Equations (2.3) and (2.4) directly. The complete program is reproduced in Appendix A.1.

```

1 def divided_diff_table(x, y):
2     """Construct divided difference table and return
3     coefficients."""
4     n = len(x)
5     table = [y[:]] # first column is y
6     for j in range(1, n):
7         col = []
8         for i in range(n - j):
9             num = table[j-1][i+1] - table[j-1][i]
10            den = x[i+j] - x[i]
11            col.append(num / den)
12        table.append(col)
13    coeffs = [table[i][0] for i in range(n)] # top-diagonal
14    # entries
15    return table, coeffs
16
17 def newton_poly(coeffs, x_points, x_val):
18     """Evaluate the Newton polynomial at x_val (Horner-like
19     nesting)."""
20     n = len(coeffs)
21     result = coeffs[-1]
22     for i in range(n-2, -1, -1):
23         result = result * (x_val - x_points[i]) + coeffs[i]
24     return result

```

Listing 2.1: Divided-difference table and Newton polynomial evaluation (extracted from question7.py).

The Lagrange form is implemented directly from Equations (2.5) and (2.6), evaluating every basis polynomial explicitly at the requested point:

```

1 def lagrange_poly(x_points, y_points, x_val):
2     """Evaluate the Lagrange polynomial at x_val."""
3     n = len(x_points)
4     result = 0.0
5     for i in range(n):
6         term = y_points[i]
7         for j in range(n):
8             if i != j:
9                 term *= (x_val - x_points[j]) / (x_points[i] -
10                x_points[j])
11         result += term
12     return result

```

Listing 2.2: Lagrange polynomial evaluation (extracted from question7.py).

The driver builds both symbolic polynomials with SymPy, evaluates each numerical routine at the requested point $x = 5$, and cross-checks the Lagrange result against SciPy's built-in lagrange function.

2.3 Results

2.3.1 Divided Difference Table

Applying Equation (2.3) to the data of Equation (2.2) produces the divided-difference table in Table 2.1. The top diagonal, read left to right, gives the Newton coefficients $f[x_0] = 3$, $f[x_0, x_1] = 3$, $f[x_0, x_1, x_2] = -\frac{7}{6}$, and $f[x_0, x_1, x_2, x_3] = \frac{4}{15}$.

Table 2.1: Divided-difference table for the data $x = (1, 2, 4, 7)$, $y = (3, 6, 5, 10)$.

x_i	$f[]$	$f[\cdot, \cdot]$	$f[\cdot, \cdot, \cdot]$	$f[\cdot, \cdot, \cdot, \cdot]$
1	3.0000	3.0000	-1.1667	0.2667
2	6.0000	-0.5000	1.6667	
4	5.0000	1.6667		
7	10.0000			

2.3.2 Newton and Lagrange Polynomials

Substituting the top-diagonal coefficients into Equation (2.4) and expanding gives the Newton interpolating polynomial in standard form,

$$P_3(x) = \frac{4}{15}x^3 - \frac{91}{30}x^2 + \frac{307}{30}x - \frac{67}{15} \approx 0.2667x^3 - 3.0333x^2 + 10.2333x - 4.4667. \quad (2.7)$$

Independently expanding the Lagrange form of Equation (2.6) for the same data gives

$$P_3(x) = 0.2667x^3 - 3.0333x^2 + 10.2333x - 4.4667, \quad (2.8)$$

which is identical, coefficient for coefficient, to Equation (2.7), exactly as the uniqueness argument of Section 1.1.4 predicts. SciPy's built-in `lagrange` routine, run on the same data as an independent cross-check, returns the same cubic.

2.3.3 Evaluation at $x = 5$

Evaluating both forms at the requested point $x = 5$ gives

$$P_3(5) = \frac{21}{5} = 4.200000, \quad (2.9)$$

with the Newton routine returning 4.199999999999999 and the Lagrange routine returning 4.199999999999998 in double precision; the two values agree to within 10^{-15} , the level of ordinary floating-point round-off, which the program's built-in verification step confirms explicitly.

2.4 Graphical Analysis

Figure 2.1 plots the Newton and Lagrange forms together with the four data points and the evaluation point $x = 5$. The two curves overlay each other exactly, visually confirming that both methods reproduce the same cubic through all four points.

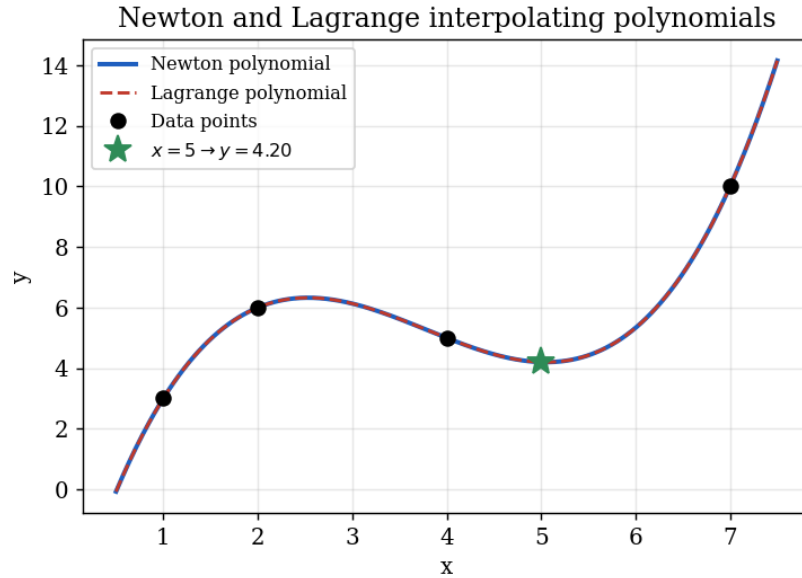


Figure 2.1: The Newton and Lagrange interpolating polynomials through the data $(1, 3)$, $(2, 6)$, $(4, 5)$, $(7, 10)$, with the evaluation point $x = 5$ marked.

Because the two curves in Figure 2.1 are visually indistinguishable, Figure 2.2 instead plots the absolute difference between the Newton and Lagrange values on a logarithmic axis over the same range of x . The difference sits at the level of machine round-off (about 10^{-13} to 10^{-15}) across the entire plotted interval, confirming numerically that the two forms are algebraically the same polynomial and that any residual discrepancy is due only to the different order of floating-point operations performed by each method.

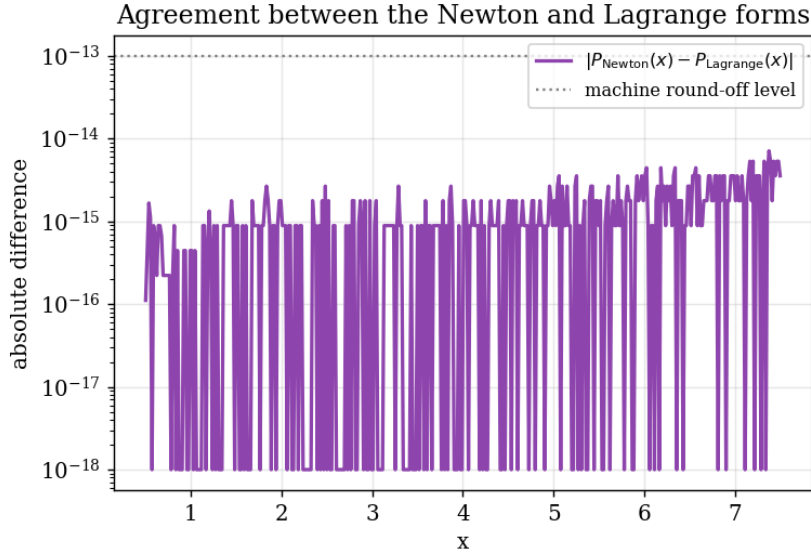


Figure 2.2: Absolute difference between the Newton and Lagrange forms evaluated over the same range of x . The difference never exceeds machine round-off, confirming the two methods construct the same interpolating polynomial.

2.5 Discussion

The experiment confirms the uniqueness theorem of Equation (2.1) in a direct and checkable way: two methods built from unrelated formulas, evaluated by different arithmetic, agree to machine precision at every point tested. The choice between the two methods in practice is therefore not about accuracy, since both are exact for the same class of interpolating polynomial, but about computational convenience. The Newton form is preferable whenever new data points may be added later, since only one new divided-difference diagonal entry is needed and the previously computed coefficients remain valid; it also evaluates efficiently using nested (Horner-like) multiplication. The Lagrange form is conceptually simpler and gives an explicit closed formula, which is convenient when the full set of nodes is fixed in advance, but every basis polynomial $L_i(x)$ must be reconstructed from scratch if the node set changes, making it less efficient for problems solved incrementally. Both forms share the same fundamental limitation of high-degree polynomial interpolation on equally or arbitrarily spaced nodes, namely that adding many additional points can make the interpolant increasingly oscillatory between nodes (Runge's phenomenon); with only four points and $x = 5$ lying inside the span of the data (interpolation rather than extrapolation), this is not a concern for the present problem.

Chapter 3

Numerical Integration of a Polynomial

This chapter addresses Question 12. It develops the theory of the two integration rules, derives the exact value of the integral, presents the Python implementation, reports the numerical results in tabular form, illustrates the behaviour of the methods graphically, and discusses the observed accuracy.

3.1 Theoretical Background

3.1.1 The Problem of Numerical Integration

Numerical integration, also called numerical quadrature, is the process of estimating the value of a definite integral

$$I = \int_a^b f(x) dx \quad (3.1)$$

using only values of the integrand $f(x)$ at a finite set of points. The underlying idea common to the classical rules is to replace the integrand on each small piece of the interval by a simple interpolating polynomial that can be integrated exactly, and then to add up the contributions of all the pieces. The interval $[a, b]$ is divided into n equal subintervals of width

$$h = \frac{b-a}{n}, \quad x_i = a + i h, \quad i = 0, 1, 2, \dots, n, \quad (3.2)$$

so that $x_0 = a$ and $x_n = b$. The rules differ in the degree of the polynomial used on each piece.

3.1.2 The Composite Trapezoidal Rule

The Trapezoidal Rule approximates the integrand on each subinterval $[x_i, x_{i+1}]$ by a straight line joining the two end values $f(x_i)$ and $f(x_{i+1})$. The area under that straight line is the area of a trapezium, $\frac{h}{2} [f(x_i) + f(x_{i+1})]$. Adding the trapezia over all n subintervals and collecting the repeated interior terms gives the composite Trapezoidal Rule

$$I_{\text{trap}} = \frac{h}{2} \left[f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right]. \quad (3.3)$$

The two end points carry weight 1 and every interior point carries weight 2. The error of the composite rule is

$$E_{\text{trap}} = -\frac{(b-a)h^2}{12} f''(\xi), \quad \xi \in (a, b), \quad (3.4)$$

so the Trapezoidal Rule is a second-order method: its error is proportional to h^2 , which means that doubling n (halving h) reduces the error by a factor of about four. Because the error depends on the second derivative f'' , the rule is exact only for polynomials of degree one or less.

3.1.3 The Composite Simpson's 1/3 Rule

Simpson's 1/3 Rule improves on the Trapezoidal Rule by fitting a parabola (a second-degree polynomial) through three consecutive points x_{i-1}, x_i, x_{i+1} instead of a straight line through two. Because each parabola spans a *pair* of subintervals, the rule requires the number of subintervals n to be **even**. Integrating each parabola exactly and summing over all pairs gives the composite Simpson's 1/3 Rule

$$I_{\text{simp}} = \frac{h}{3} \left[f(x_0) + 4 \sum_{\substack{i=1 \\ i \text{ odd}}}^{n-1} f(x_i) + 2 \sum_{\substack{i=2 \\ i \text{ even}}}^{n-2} f(x_i) + f(x_n) \right]. \quad (3.5)$$

The two end points carry weight 1, the odd-indexed interior points carry weight 4, and the even-indexed interior points carry weight 2. The error of the composite rule is

$$E_{\text{simp}} = -\frac{(b-a)h^4}{180} f^{(4)}(\xi), \quad \xi \in (a, b), \quad (3.6)$$

so Simpson's 1/3 Rule is a fourth-order method: its error is proportional to h^4 , and it depends on the *fourth* derivative $f^{(4)}$. A key consequence is that Simpson's 1/3 Rule is exact for every polynomial of degree three or less, because the fourth derivative of any cubic is identically zero. This fact is important for the present problem and is confirmed by the results below.

3.1.4 Exact Value of the Integral

The integrand in this problem, $f(x) = x^3 + 2x + 1$, has an elementary antiderivative, so the exact value can be found analytically and used as the reference. Integrating term by term,

$$\int (x^3 + 2x + 1) dx = \frac{x^4}{4} + x^2 + x + C, \quad (3.7)$$

and evaluating between the limits 0 and 2,

$$I = \left[\frac{x^4}{4} + x^2 + x \right]_0^2 = \left(\frac{16}{4} + 4 + 2 \right) - (0) = 4 + 4 + 2 = 10. \quad (3.8)$$

The exact value of the integral is therefore $I = 10$. Since the integrand is a cubic, Equation (3.6) predicts that Simpson's 1/3 Rule should return exactly 10 (up to machine round-off) for *any* even n , whereas the Trapezoidal Rule should return a value that approaches 10 only as n grows.

3.2 Python Implementation

The program was written in Python 3 using NumPy for vectorised array arithmetic and SymPy for the exact (symbolic) value of the integral, so that the same driver would report the correct reference automatically if the integrand or limits were changed. Listing 3.1 shows the two integration routines, which implement Equations (3.3) and (3.5) directly using NumPy array slicing rather than explicit Python loops: the endpoint terms are added once and the interior points are summed with `numpy.sum` over the appropriate stride. The complete program, including the symbolic exact value and the tabular output, is reproduced in Appendix A.2.

```
1 def f(x):
2     return x**3 + 2*x + 1
3
4 def trapezoidal_rule(a, b, n):
5     h = (b - a) / n
6     x = np.linspace(a, b, n+1)
7     y = f(x)
8     integral = 0.5 * (y[0] + y[-1]) + np.sum(y[1:-1])
9     return integral * h
10
11 def simpson_rule(a, b, n):
12     if n % 2 != 0:
13         raise ValueError("n must be even for Simpson's 1/3 rule")
14     h = (b - a) / n
15     x = np.linspace(a, b, n+1)
16     y = f(x)
17     integral = y[0] + y[-1]
18     integral += 4 * np.sum(y[1:-1:2])
19     integral += 2 * np.sum(y[2:-1:2])
20     return integral * h / 3
```

Listing 3.1: The two numerical integration rules (extracted from `question12.py`).

The exact value is obtained symbolically rather than hard-coded, using SymPy's `integrate` on the exact algebraic form of the integrand:

```
1 def exact_integral_sym():
2     x = sp.Symbol('x')
3     expr = x**3 + 2*x + 1
4     integral = sp.integrate(expr, (x, 0, 2))
5     return float(integral), integral
```

Listing 3.2: Exact value of the integral, computed symbolically.

The driver reads n from the user and validates that it is a positive integer; if n is odd, the Trapezoidal Rule is still evaluated but Simpson's 1/3 Rule is skipped and reported as not applicable, since Equation (3.5) requires an even number of subintervals.

3.3 Results and Output

Running the program with $n = 8$ subintervals produces the console output shown in Listing 3.3. The Trapezoidal Rule overestimates the integral by 0.0625, while Simpson's 1/3 Rule returns the exact value 10 as predicted.

```
Enter number of subintervals (even for Simpson): 8

Exact integral (symbolic): 10
Exact numeric value: 10.0000000000

=====
Numerical Integration Results
=====
Method                Approximation          Absolute Error
-----
Trapezoidal Rule      10.0625000000          0.0625000000
Simpson 1/3 Rule      10.0000000000          0.0000000000
=====
```

Listing 3.3: Program output for $n = 8$.

To study how the accuracy depends on the number of subintervals, the program was run for a range of even values of n ; the results are collected in Table 3.1. The Trapezoidal error falls by almost exactly a factor of four each time n is doubled, which is the signature of the second-order $O(h^2)$ behaviour predicted by Equation (3.4). Simpson's 1/3 Rule is exact for this cubic at every n , exactly as Equation (3.6) predicts.

Table 3.1: Absolute error of the two rules for the integral $\int_0^2 (x^3 + 2x + 1) dx$ (exact value = 10).

n	Trapezoidal value	Trap. error	Simpson value	Simpson error
2	11.0000000000	1.000×10^0	10.0000000000	0
4	10.2500000000	2.500×10^{-1}	10.0000000000	0
8	10.0625000000	6.250×10^{-2}	10.0000000000	0
16	10.0156250000	1.562×10^{-2}	10.0000000000	0
32	10.0039062500	3.906×10^{-3}	10.0000000000	0
64	10.0009765625	9.766×10^{-4}	10.0000000000	0

3.4 Graphical Analysis

Two figures are used to illustrate the results. Figure 3.1 shows the integrand $f(x) = x^3 + 2x + 1$ on $[0, 2]$ together with the four trapezia used when $n = 4$. The straight tops of the trapezia lie above the curve wherever the curve is concave, which is why the Trapezoidal Rule overestimates the area for this convex integrand, consistent with the positive errors in Table 3.1.

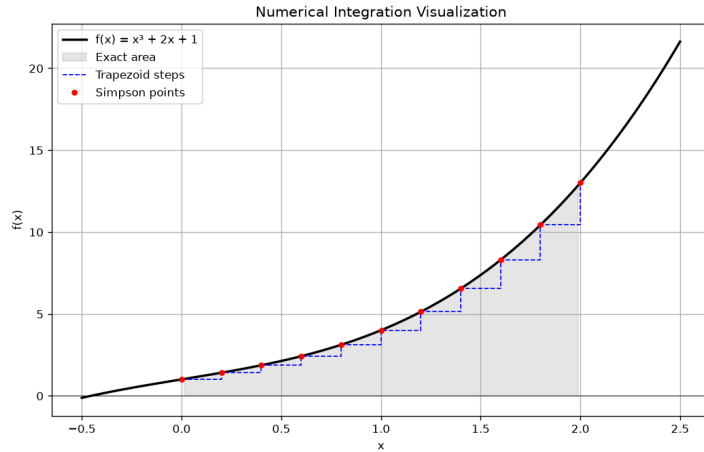


Figure 3.1: The integrand $f(x) = x^3 + 2x + 1$ and the trapezoidal approximation of the area for $n = 4$. The trapezium tops lie above the convex curve, so the rule overestimates the true area.

Figure 3.2 plots the absolute error against n on logarithmic axes. The Trapezoidal error follows a straight line of slope -2 , confirming the $O(h^2)$ convergence, and lies parallel to the reference line $\propto n^{-2}$. The Simpson error is not shown as a genuine trend because it is zero in exact arithmetic; the values plotted for it sit at the level of floating-point round-off (about 10^{-15}), which is why the curve is flat and far below the Trapezoidal line.

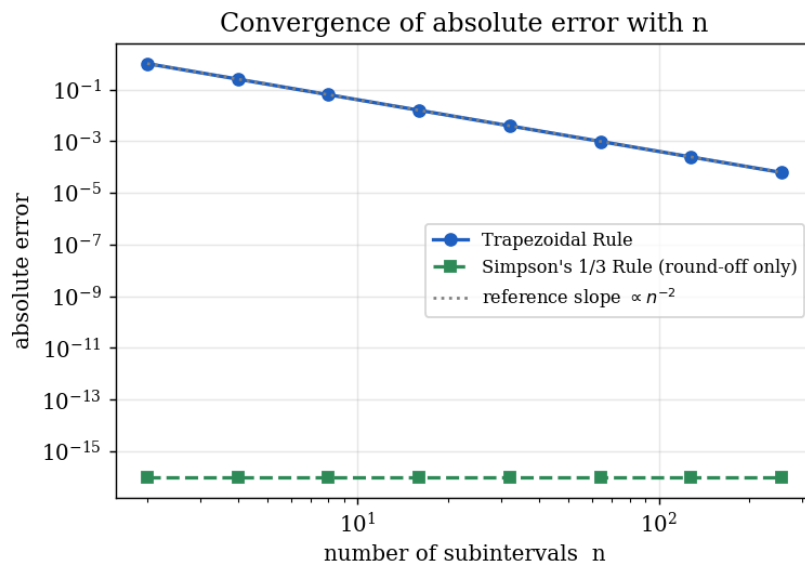


Figure 3.2: Absolute error versus the number of subintervals n on log-log axes. The Trapezoidal error decreases with slope -2 ($O(h^2)$); Simpson's error stays at the machine round-off floor because the rule is exact for cubics.

3.5 Discussion

The experiment confirms the theory in a very clean way. Simpson's 1/3 Rule integrates the cubic integrand exactly because the rule is exact for all polynomials up to degree three, and its residual error of order 10^{-15} is simply the accumulation of floating-point round-off rather than any shortcoming of the method. The Trapezoidal Rule, being exact only for straight lines, leaves a genuine truncation error that shrinks as $O(h^2)$; the factor-of-four reduction per doubling of n seen in Table 3.1 is an unusually tidy confirmation of that order. For a smooth integrand such as this one, Simpson's 1/3 Rule is therefore clearly preferable, delivering far higher accuracy for the same number of function evaluations. The only practical constraint is that Simpson's rule requires an even n ; the program checks this explicitly and reports Simpson's rule as not applicable for an odd input rather than silently adjusting it.

Chapter 4

Conclusion

This report set out to study, implement, and analyse four classical numerical methods on two assigned problems, each of which admits an independent check on the computed result. All of the objectives stated in Section 1.3 were met: the theory of each method was derived, all four methods were implemented in Python 3 using NumPy, SciPy, and SymPy, every numerical result was checked against an independent reference, and the behaviour of each method was made visible through clearly labelled graphs.

For the interpolation problem, Newton's Divided Difference method and the Lagrange method were applied to the same four data points and, as the uniqueness theorem for polynomial interpolation predicts, produced algebraically identical cubic polynomials, $P_3(x) = \frac{4}{15}x^3 - \frac{91}{30}x^2 + \frac{307}{30}x - \frac{67}{15}$. Evaluating both forms at $x = 5$ gave the same value, 4.2, to within machine round-off, and SciPy's built-in Lagrange routine confirmed the same cubic independently. The two methods therefore do not differ in accuracy at all; they differ only in computational convenience, with the Newton form better suited to problems where data points are added incrementally, and the Lagrange form offering a simpler, explicit closed form when the full node set is fixed in advance.

For the numerical integration problem, the results confirmed the theory precisely. Simpson's 1/3 Rule returned the exact value 10 for the cubic integrand at every choice of n , because the rule is exact for polynomials up to degree three, while the Trapezoidal Rule left a genuine truncation error that fell by a factor of four each time the number of subintervals was doubled, the unmistakable signature of second-order $O(h^2)$ accuracy. For a smooth integrand, Simpson's 1/3 Rule is clearly the better choice, delivering far higher accuracy for the same number of function evaluations.

Taken together, the two problems make a single broader point clear: understanding a numerical method means understanding not just how to compute an answer but also how to check it, whether that check comes from an alternative method that must agree by a uniqueness argument, from a known exact value, or from an independent library routine. It also shows that different methods for the same task are not always in competition on accuracy alone; sometimes, as with Newton and Lagrange interpolation, the choice is purely a matter of computational convenience, while in other cases, as with the Trapezoidal Rule and Simpson's 1/3 Rule, the choice of method has a direct and measurable effect on the accuracy obtained for a given amount of work.

4.1 Future Work

Several natural extensions would build on this study. The interpolation comparison could be repeated with additional, more widely or unevenly spaced data points to study the onset of Runge's phenomenon, and with an evaluation point outside the

span of the data to examine the reliability (or unreliability) of polynomial extrapolation. The integration comparison could be repeated with an integrand that has no elementary antiderivative and a non-zero fourth derivative, so that Simpson's rule also exhibits a measurable $O(h^4)$ error trend rather than being exact; an adaptive quadrature scheme that places more points where the integrand varies rapidly could then be compared against the fixed-step rules. Finally, wrapping the routines in a small automated test harness would let the accuracy and agreement claims be re-verified automatically whenever the code is changed.

References

- Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585:357–362.
- Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science and Engineering*, 9(3):90–95.
- Python Software Foundation (2024). The Python language reference. Retrieved from <https://docs.python.org/3/>.

Appendix A: Complete Python Source Code

This appendix reproduces the two complete Python programs exactly as they were written and run to produce the results and figures in this report. Both programs use NumPy for array computation, SymPy for exact symbolic results, and Matplotlib for plotting; Question 7 additionally uses SciPy's `scipy.interpolate.lagrange` as an independent cross-check.

A.1 Polynomial Interpolation Program (`question7.py`)

This program answers Question 7. It builds the divided-difference table for the data $x = (1, 2, 4, 7)$, $y = (3, 6, 5, 10)$, constructs the Newton and Lagrange interpolating polynomials symbolically with SymPy, evaluates both at $x = 5$, verifies that the two values agree, cross-checks against SciPy's built-in Lagrange routine, and plots both polynomials together with the data.

```
1  """
2  Question 7: Newton and Lagrange Interpolation (with libraries)
3  Data: X = [1, 2, 4, 7], Y = [3, 6, 5, 10]
4  Uses NumPy, SciPy, SymPy, Matplotlib.
5  """
6
7  import numpy as np
8  import sympy as sp
9  import matplotlib.pyplot as plt
10 from scipy.interpolate import lagrange # for comparison
11
12 def divided_diff_table(x, y):
13     """Construct divided difference table and return
14     coefficients."""
15     n = len(x)
16     table = [y[:]] # first column is y
17     for j in range(1, n):
18         col = []
19         for i in range(n - j):
20             num = table[j-1][i+1] - table[j-1][i]
21             den = x[i+j] - x[i]
22             col.append(num / den)
23         table.append(col)
24     coeffs = [table[i][0] for i in range(n)]
25     return table, coeffs
26
27 def newton_poly(coeffs, x_points, x_val):
28     """Evaluate Newton polynomial at x_val."""
29     n = len(coeffs)
30     result = coeffs[-1]
31     for i in range(n-2, -1, -1):
32         result = result * (x_val - x_points[i]) + coeffs[i]
33     return result
34
35 def lagrange_poly(x_points, y_points, x_val):
```

```

35     """Evaluate Lagrange polynomial at x_val."""
36     n = len(x_points)
37     result = 0.0
38     for i in range(n):
39         term = y_points[i]
40         for j in range(n):
41             if i != j:
42                 term *= (x_val - x_points[j]) / (x_points[i] -
43                                                    x_points[j])
44         result += term
45     return result
46
47 def main():
48     # Data
49     x = np.array([1, 2, 4, 7], dtype=float)
50     y = np.array([3, 6, 5, 10], dtype=float)
51
52     # 1. Divided difference table and coefficients
53     table, coeffs = divided_diff_table(x, y)
54     print("Divided Difference Table")
55     print("-----")
56     header = "x_i\tf[ ]"
57     for i in range(1, len(x)):
58         header += f"\tf[...,{i}]"
59     print(header)
60     for i in range(len(x)):
61         row = f"{x[i]:.0f}\t"
62         for j in range(len(x) - i):
63             row += f"{table[j][i]:.4f}\t"
64         print(row)
65     print()
66
67     # 2. Newton polynomial (symbolic)
68     X = sp.Symbol('x')
69     newton_expr = coeffs[0]
70     prod = 1
71     for i in range(1, len(coeffs)):
72         prod *= (X - x[i-1])
73         newton_expr += coeffs[i] * prod
74     print("Newton General Interpolation Polynomial (symbolic):")
75     sp.pprint(newton_expr, use_unicode=True)
76     print()
77
78     # 3. Evaluate at x=5 using Newton
79     x_eval = 5
80     newton_val = newton_poly(coeffs, x, x_eval)
81     print(f"Newton evaluation at x = {x_eval}: {newton_val:.6f}")
82
83     # 4. Lagrange evaluation at x=5
84     lagrange_val = lagrange_poly(x, y, x_eval)
85     print(f"Lagrange evaluation at x = {x_eval}: {lagrange_val:.6f}")

```

```

86     # 5. Verify
87     print("\nVerification:")
88     if np.isclose(newton_val, lagrange_val, rtol=1e-9):
89         print("✓ Both methods give the same value at x = 5.")
90     else:
91         print("✗ Values differ!")
92
93     # 6. Display Lagrange polynomial (symbolic)
94     # Build Lagrange basis symbolically
95     lagrange_expr = 0
96     for i in range(len(x)):
97         basis = 1
98         for j in range(len(x)):
99             if i != j:
100                 basis *= (X - x[j]) / (x[i] - x[j])
101             lagrange_expr += y[i] * basis
102     print("\nLagrange Interpolation Polynomial (symbolic):")
103     sp.pprint(sp.expand(lagrange_expr), use_unicode=True) #
104         expand for clarity
105
106     # (Optional) Compare with SciPy's built-in Lagrange
107     scipy_poly = lagrange(x, y)
108     scipy_val = scipy_poly(x_eval)
109     print(f"\nSciPy Lagrange evaluation at x = {x_eval}: {
110         scipy_val:.6f}")
111     print("(Should match our Lagrange value)")
112
113     # Plotting
114     x_plot = np.linspace(0, 8, 200)
115     # Newton function (vectorized)
116     def newton_func(xx):
117         return newton_poly(coeffs, x, xx)
118     y_newton = np.array([newton_func(xx) for xx in x_plot])
119     y_lagrange = lagrange_poly(x, y, x_plot) # our function
120         works with arrays
121
122     plt.figure(figsize=(10, 6))
123     plt.plot(x_plot, y_newton, 'b-', label='Newton polynomial')
124     plt.plot(x_plot, y_lagrange, 'r--', label='Lagrange
125         polynomial')
126     plt.plot(x, y, 'ko', markersize=8, label='Data points')
127     plt.plot(x_eval, newton_val, 'g*', markersize=12, label=f'x
128         ={x_eval}')
129     plt.xlabel('x')
130     plt.ylabel('y')
131     plt.title('Newton and Lagrange Interpolation')
132     plt.legend()
133     plt.grid(True)
134     plt.show()
135
136     if __name__ == "__main__":
137         main()

```

Listing 4.1: Complete source of question7.py.

A.2 Numerical Integration Program (question12.py)

This program answers Question 12. It reads the number of subintervals from the user, evaluates $\int_0^2 (x^3 + 2x + 1) dx$ by the Trapezoidal Rule and Simpson's 1/3 Rule, computes the exact value symbolically with SymPy, prints the approximations and absolute errors in a table, and plots the integrand together with the trapezoidal approximation.

```
1  """
2  Question 12: Numerical Integration with libraries
3  Evaluate  $\int_0^2 (x^3 + 2x + 1) dx$ 
4  Uses NumPy, SymPy, Matplotlib.
5  """
6
7  import numpy as np
8  import sympy as sp
9  import matplotlib.pyplot as plt
10
11 def f(x):
12     return x**3 + 2*x + 1
13
14 def exact_integral_sym():
15     x = sp.Symbol('x')
16     expr = x**3 + 2*x + 1
17     integral = sp.integrate(expr, (x, 0, 2))
18     return float(integral), integral
19
20 def trapezoidal_rule(a, b, n):
21     h = (b - a) / n
22     x = np.linspace(a, b, n+1)
23     y = f(x)
24     integral = 0.5 * (y[0] + y[-1]) + np.sum(y[1:-1])
25     return integral * h
26
27 def simpson_rule(a, b, n):
28     if n % 2 != 0:
29         raise ValueError("n must be even for Simpson's 1/3 rule")
30     h = (b - a) / n
31     x = np.linspace(a, b, n+1)
32     y = f(x)
33     integral = y[0] + y[-1]
34     integral += 4 * np.sum(y[1:-1:2])
35     integral += 2 * np.sum(y[2:-1:2])
36     return integral * h / 3
37
38 def main():
39     # Input
40     while True:
41         try:
42             n = int(input("Enter number of subintervals (even for Simpson): "))
43             if n <= 0:
44                 print("n must be positive.")
```

```

45         continue
46     break
47 except ValueError:
48     print("Enter an integer.")
49
50 a, b = 0, 2
51
52 # Exact value using SymPy (also analytical)
53 exact_val, exact_sym = exact_integral_sym()
54 print(f"\nExact integral (symbolic): {exact_sym}")
55 print(f"Exact numeric value: {exact_val:.10f}")
56
57 # Numerical methods
58 trap_val = trapezoidal_rule(a, b, n)
59 trap_err = abs(exact_val - trap_val)
60
61 try:
62     simp_val = simpson_rule(a, b, n)
63     simp_err = abs(exact_val - simp_val)
64 except ValueError as e:
65     simp_val = None
66     simp_err = None
67     print(e)
68
69 # Results table
70 print("\n" + "="*60)
71 print("Numerical Integration Results")
72 print("="*60)
73 print(f"{'Method':<20} {'Approximation':<20} {'Absolute Error':<20}")
74 print("-"*60)
75 print(f"{'Trapezoidal Rule':<20} {trap_val:<20.10f} {trap_err:<20.10f}")
76 if simp_val is not None:
77     print(f"{'Simpson 1/3 Rule':<20} {simp_val:<20.10f} {simp_err:<20.10f}")
78 else:
79     print(f"{'Simpson 1/3 Rule':<20} {'Not applicable':<20} {'---':<20}")
80 print("="*60)
81
82 # Plot the function and the approximations
83 x_plot = np.linspace(a-0.5, b+0.5, 200)
84 y_plot = f(x_plot)
85
86 plt.figure(figsize=(10, 6))
87 plt.plot(x_plot, y_plot, 'k-', linewidth=2, label=f'f(x) = x3 + 2x + 1')
88 plt.fill_between(x_plot, 0, y_plot, where=(x_plot>=a)&(x_plot<=b), alpha=0.2, color='gray', label='Exact area')
89
90 # Show trapezoids (if n not too large for visibility)
91 if n <= 20:
92     x_trap = np.linspace(a, b, n+1)

```



```

93     y_trap = f(x_trap)
94     plt.step(x_trap, y_trap, where='post', color='blue',
95             linestyle='--', linewidth=1, label='Trapezoid steps
96             ')
97     # Simpson not easy to show via step, but we can show
98     points
99     if simp_val is not None:
100         x_simp = np.linspace(a, b, n+1)
101         y_simp = f(x_simp)
102         plt.plot(x_simp, y_simp, 'ro', markersize=4, label=
103                 'Simpson points')
104
105     plt.xlabel('x')
106     plt.ylabel('f(x)')
107     plt.title('Numerical Integration Visualization')
108     plt.axhline(0, color='black', linewidth=0.5)
109     plt.legend()
110     plt.grid(True)
111     plt.show()
112
113 if __name__ == "__main__":
114     main()

```

Listing 4.2: Complete source of question12.py.